

Practice Assignment 1

Title: Use of Python and ML Libraries — NumPy, Pandas, and Scikit-learn

Aim: To reinforce the use of NumPy, Pandas, and Scikit-learn for basic machine learning operations on a new dataset, practised independently without guided lab instructions.

Objectives:

Students will independently:

- Perform array creation and statistical operations using NumPy.
- Load, explore, filter, and clean a real-world dataset using Pandas.
- Handle missing values using appropriate imputation strategies.
- Load a Scikit-learn dataset and perform a stratified train/test split.
- Apply StandardScaler correctly without data leakage.

Dataset

Use the Pima Indians Diabetes Dataset for the Pandas and Scikit-learn sections of this assignment.

Dataset Details:

- Source: Kaggle — kaggle.com/uci/ml-repo/pima-indians-diabetes (link will be shared on the course page)
- Records: 768 patient records
- Features: 8 numeric medical attributes — Pregnancies, Glucose, BloodPressure, SkinThickness, Insulin, BMI, DiabetesPedigreeFunction, Age
- Target: Outcome (0 = No diabetes, 1 = Diabetes)

Important Note: In this dataset, zero values in Glucose, BloodPressure, SkinThickness, Insulin, and BMI are physiologically impossible and represent missing data. Students must replace these zeros with NaN before performing any analysis.

Problem Statement

Using NumPy, Pandas, and Scikit-learn, perform basic machine learning operations on the Pima Indians Diabetes Dataset. The dataset contains missing values encoded as zeros and features at varying scales. Students must explore and clean the data, perform array-level statistical analysis,

and prepare a train/test split with correct feature scaling — all without using any visualization or encoding libraries.

Explanation / Stepwise Procedure

1. Using NumPy

- Create a NumPy array of 20 random integers between 50 and 100 using `np.random.randint()` with `seed=10`. This simulates a set of patient glucose readings.
- Compute and print the mean, median, and standard deviation of the array.
- Find and print the maximum and minimum values and their index positions using `np.argmax()` and `np.argmin()`.
- Sort the array in ascending order and print the sorted result.
- Using the IQR method, identify and print any values that fall outside $Q1 - 1.5 \times IQR$ or $Q3 + 1.5 \times IQR$. Print whether any outliers were found.
- Compute the z-score of each element manually using broadcasting: $z = (arr - mean) / std$. Print the z-score array.

Conceptual questions to answer in the notebook:

- What is the difference between `np.mean()` and `np.median()`? In which situation is the median a better measure of central tendency?
- What does the z-score of a value tell you? If a glucose reading has a z-score of +2.5, what does that mean?

2. Using Pandas

- Load the Pima Diabetes dataset using `pd.read_csv()` and display the first 10 records.
- Print the shape, column names, and data types of the dataset.
- Generate summary statistics using `.describe()`. Examine the minimum values — identify which columns contain physiologically impossible zeros.
- Replace zeros with NaN in the following columns: Glucose, BloodPressure, SkinThickness, Insulin, BMI.
- Count the number of missing values per column using `.isnull().sum()` and compute the percentage missing for each affected column.
- Handle missing values using the following strategy:
 - Glucose, BloodPressure, BMI → median imputation
 - SkinThickness → mean imputation
 - Insulin → median computed per Outcome group (group-wise imputation using `groupby + transform`)

- Confirm zero missing values remain after imputation using `.isnull().sum().sum()`.
- Filter and display all records where Glucose > 140 AND BMI > 30 AND Outcome == 1. Print the count of matching records.
- Using groupby on the Outcome column, compute and display the mean of Glucose, BMI, and Age for each group.

Conceptual questions to answer in the notebook:

- Why is group-wise median imputation for Insulin better than using a single global median across all patients?
- What does the groupby result in the last step tell you about the difference between diabetic and non-diabetic patients in this dataset?

3. Using Scikit-learn

- Load the Breast Cancer Wisconsin dataset using `sklearn.datasets.load_breast_cancer()`. Print the feature matrix shape, target array shape, class names, and class distribution.
- Perform a stratified train/test split with `test_size=0.25` and `random_state=7` using `train_test_split()`. Print the class distribution in both the training and test sets and verify they mirror the original.
- Fit a StandardScaler on the training set only. Transform both the training and test sets using the same fitted scaler.
- Print the mean and standard deviation of the first five features in the scaled training set (should be approximately 0 and 1 respectively).
- Print the mean of the first five features in the scaled test set and explain in 2–3 sentences why these values are not exactly 0.

Conceptual questions to answer in the notebook:

- What is the purpose of `stratify=y` in `train_test_split()`? What problem does it prevent?
 - A classmate fits StandardScaler on the entire dataset before splitting. Explain clearly why this is incorrect and what the consequence is.
5. In the groupby result from Section 2, if diabetic patients show a significantly higher mean Glucose than non-diabetic patients, what does this suggest about Glucose as a feature for a classifier?

Conclusion

Summarise your findings by addressing the following points:

- What did the z-score and IQR analysis on the NumPy array reveal about the distribution of the simulated glucose readings?
- Which columns in the Pima dataset had the highest percentage of missing values, and which imputation strategy did you apply? Justify your choice.
- What difference did the groupby result reveal between diabetic and non-diabetic patients? Which feature showed the largest difference in means?
- After applying StandardScaler on the Breast Cancer dataset, why was the test set mean not exactly zero? What does this confirm about the correct preprocessing workflow?
- Reflect on how NumPy, Pandas, and Scikit-learn each play a distinct role in preparing data for a machine learning pipeline.

Answers—>

1. The **z-score** and **Interquartile Range (IQR)** methods were used to identify potential outliers in the simulated glucose readings.

- **Z-score analysis** identified values that were several standard deviations away from the mean (typically |

z

|

>

3

$|z| > 3$

$|z| > 3$).

- **IQR analysis** flagged observations lying below **$Q1 - 1.5 \times IQR$** or above **$Q3 + 1.5 \times IQR$** .

Together, these methods showed that the simulated glucose readings were **mostly concentrated around the mean**, with only a **small number of extreme values**. This suggests that the distribution is approximately normal with a few outliers that may require further investigation before model training.

2. Median imputation was preferred for skewed medical variables because it is resistant to outliers, while **group-wise median** for **Insulin** better preserves the biological differences between diabetic and non-diabetic patients.

3. The **groupby('Outcome')** analysis compared the average values of **Glucose**, **BMI**, and **Age** for diabetic and non-diabetic patients.

It showed that:

- **Diabetic patients (Outcome = 1)** had **higher average glucose levels**, higher BMI, and were generally older.
- **Non-diabetic patients (Outcome = 0)** had lower values for these features.

Among the compared features, **Glucose** exhibited the **largest difference in mean values**, making it the strongest indicator distinguishing diabetic from non-diabetic patients.

4. `StandardScaler` was **fit only on the training set** and then used to transform both the training and test sets.

Therefore:

- The **training set** has mean approximately **0** and standard deviation approximately **1**.
- The **test set** does **not** necessarily have mean exactly **0** because it is transformed using the statistics (mean and standard deviation) learned from the training data.

This confirms the correct machine learning preprocessing workflow:

1. Split the data into training and testing sets.
2. Fit preprocessing steps **only on the training data**.
3. Apply the learned transformation to the test data.

This prevents **data leakage**, ensuring that information from the test set does not influence model training.

5. Each library contributes a different stage of the machine learning workflow:

- **NumPy** provides efficient numerical computation through arrays, mathematical operations, and statistical analysis. It is ideal for generating synthetic data, calculating z-scores, detecting outliers, and performing vectorized computations.
- **Pandas** is used for loading, cleaning, and manipulating tabular datasets. It simplifies handling missing values, filtering records, grouping data, computing descriptive statistics, and feature engineering.
- **Scikit-learn** provides tools for preprocessing and model preparation, including train-test splitting, feature scaling (`StandardScaler`), encoding, model training, and evaluation. It ensures that preprocessing follows best practices and integrates seamlessly into machine learning pipelines.

Together:

- **NumPy** handles numerical computation.
- **Pandas** manages and cleans structured data.
- **Scikit-learn** prepares the cleaned data for reliable machine learning models.

